

Modelo Lógico: banco SIGLab

Matheus Henrique Menezes da Silva

Primeira decisão

Acredito que devemos separar o modelo em quatro módulos.

None

Usuários

Equipamentos

Solicitações

Empréstimos (v1.1)

Mesmo que o empréstimo não faça parte do MVP, eu já o deixaria previsto.

1. Usuário

None

USUARIO

id (PK)

nome

email

senha

perfil

ativo

created_at

updated_at

Observações

email

Unique.

Será utilizado para login.

perfil

Enum.

None

USER

ADMIN

Não criaria tabela Perfil.

São apenas dois valores.

2. Equipamento

Aqui existe uma decisão importante.

Eu faria assim:

None

EQUIPAMENTO

```
id (PK)

nome

descricao

categoria

patrimonio

estoque_total

ativo

created_at

updated_at
```

Observe que retirei

```
None
estoque_disponivel
```

Conforme discutimos anteriormente.

Também acrescentaria

```
None
patrimonio
```

Porque laboratório normalmente identifica equipamentos por patrimônio.

Mesmo que existam cinco notebooks iguais.

Categoria

Aqui existem duas possibilidades.

Opção A

None

`categoria VARCHAR`

Muito simples.

Opção B

None

`CATEGORIA`

`id`

`nome`

Eu iria de **Categoria como entidade**.

Por quê?

Imagine daqui um ano.

O administrador deseja criar

- Drone
- Osciloscópio
- Impressora 3D

Sem alterar o sistema.

Então eu faria:

None

`Categoria`

`1`

↓

N

Equipamentos

Solicitação

Esta é a tabela principal.

None

SOLICITACAO

id (PK)

usuario_id (FK)

equipamento_id (FK)

administrador_id (FK)

status

finalidade

data_solicitacao

data_resposta

motivo_resposta

created_at

updated_at

Observe uma coisa importante.

Existem duas FKs para Usuário.

None

`usuario_id`

↓

`quem solicitou`

e

None

`administrador_id`

↓

`quem analisou`

É uma solução bastante comum.

Histórico

Eu faria uma pequena mudança.

None

`EVENTO_HISTORICO`

`id (PK)`

`solicitacao_id (FK)`

`usuario_id (FK)`

`tipo_evento`

`descricao`

created_at

Observe.

Não armazenamos

None

status anterior

status novo

Porque isso é específico.

No futuro teremos

None

RETIRADA

DEVOLUÇÃO

MANUTENÇÃO

RENOVAÇÃO

Então eu faria algo mais genérico.

tipo_evento

None

CRIADA

APROVADA

REJEITADA

RETIRADA

DEVOLVIDA

OBSERVACAO

Empréstimo

Mesmo não implementando agora.

None

EMPRESTIMO

id

solicitacao_id

data_retirada

data_prevista

data_devolucao

status

created_at

updated_at

Relacionamento

None

Solicitação

1 ----- 0..1

Empréstimo

Relacionamentos

None	USUARIO	1	-----	N	SOLICITACAO
None	USUARIO	1	-----	N	EVENTO_HISTORICO
None	USUARIO	1	-----	N	SOLICITACAO
	(administra)				
None	CATEGORIA	1	-----	N	EQUIPAMENTO
None	EQUIPAMENTO	1	-----	N	SOLICITACAO
None	SOLICITACAO	1	-----	N	EVENTO_HISTORICO

None

SOLICITACAO

1 ----- 0..1 EMPRESTIMO

Agora vem uma decisão arquitetural

Existe uma tabela que eu acrescentaria.

MOVIMENTACAO_ESTOQUE

Hoje parece desnecessária.

Mas pense.

None

Notebook

Quantidade

10

Depois de um ano.

Quem alterou?

Quando?

Por quê?

Não sabemos.

Então criaria.

None

MOVIMENTACAO_ESTOQUE

id

equipamento_id

usuario_id

tipo

quantidade

motivo

created_at

Exemplo

None

ENTRADA

SAIDA

MANUTENCAO

DESCARTE

AJUSTE

Isso resolve completamente auditoria de estoque.

Minha sugestão para o MVP

Eu simplificaria um pouco mais, para reduzir o tempo de desenvolvimento.

MVP (v1.0)

None

USUARIO

id

nome
email
senha
perfil
ativo

None
CATEGORIA

id
nome

None
EQUIPAMENTO

id
categoria_id
nome
descricao
patrimonio
estoque_total
ativo

None
SOLICITACAO

id
usuario_id
equipamento_id
administrador_id

status

finalidade

data_solicitacao

`data_resposta`

`motivo_resposta`

None

`EVENTO_HISTORICO`

`id`

`solicitacao_id`

`usuario_id`

`tipo_evento`

`descricao`

`created_at`

Esse modelo é enxuto, cobre todos os requisitos do MVP e deixa espaço para evolução.

Minha única recomendação antes de criar o DER

Antes de abrir qualquer ferramenta como **BrModelo**, **Draw.io** ou **MySQL Workbench**, eu faria um **Dicionário de Dados**. Para cada atributo, definiríamos:

- nome da coluna;
- tipo de dado (UUID, BIGINT, VARCHAR, TIMESTAMP etc.);
- obrigatoriedade (**NOT NULL** ou opcional);
- restrições (**UNIQUE**, **CHECK**, valores permitidos);
- valor padrão, quando existir;
- descrição da regra de negócio associada.

Isso evita ambiguidades e facilita muito a implementação posterior em **Spring Boot**, **JPA/Hibernate** e **PostgreSQL**. É um passo frequentemente ignorado em projetos acadêmicos, mas muito utilizado em projetos profissionais.

me gere um exemplo desse dicionário

Excelente ideia. O dicionário de dados é um documento que fica entre a modelagem conceitual e a implementação. Ele responde perguntas como:

- O que esse atributo representa?
- Qual o tipo de dado?
- É obrigatório?
- Pode repetir?
- Existe alguma regra de negócio sobre ele?

Vou montar um exemplo como se estivéssemos documentando o SIGLab em uma empresa.

Dicionário de Dados — SIGLab MVP

Entidade: USUARIO

Campo	Tipo	Obrigatório	Chave	Regra	Descrição
id	UUID	Sim	PK	Gerado pelo sistema	Identificador único do usuário
nome	VARCHAR(150)	Sim			Nome completo do usuário
email	VARCHAR(150)	Sim	UNIQUE	Deve ser único	Utilizado para autenticação
senha	VARCHAR(255)	Sim		Senha criptografada	Nunca armazenar senha em texto puro
perfil	ENUM	Sim		USER ou ADMIN	Define permissões do usuário
ativo	BOOLEAN	Sim		Default TRUE	Indica se o usuário pode acessar o sistema
created_at	TIMESTAMP	Sim		Gerado automaticamente	Data de criação
updated_at	TIMESTAMP	Sim		Atualizado automaticamente	Última alteração

Regras

- Email não pode repetir.
- Apenas usuários ativos podem autenticar.
- Apenas usuários com perfil ADMIN podem analisar solicitações.

Entidade: CATEGORIA

Campo	Tipo	Obrigatório	Chave	Regra	Descrição
id	UUID	Sim	PK	Gerado automaticamente	Identificador da categoria
nome	VARCHAR(100)	Sim	UNIQUE	Não repetir	Nome da categoria

Exemplos

- Notebook
- Osciloscópio
- Arduino
- Sensor
- FPGA

Entidade: EQUIPAMENTO

Campo	Tipo	Obrigatório	Chave	Regra	Descrição
id	UUID	Sim	PK		Identificador
categoria_id	UUID	Sim	FK	Deve existir categoria	Categoria do equipamento
nome	VARCHAR(150)	Sim			Nome do equipamento
descricao	TEXT	Não			Descrição detalhada

patrimonio	VARCHAR(50)	Não	UNIQUE	Opcional no MVP	Número patrimonial
estoque_total	INTEGER	Sim		Maior que zero	Quantidade disponível no laboratório
ativo	BOOLEAN	Sim		Default TRUE	Indica disponibilidade para solicitação
created_at	TIMESTAMP	Sim			Data de cadastro
updated_at	TIMESTAMP	Sim			Última alteração

Regras

- estoque_total > 0
- Equipamentos inativos não podem ser solicitados.

Entidade: SOLICITACAO

Campo	Tipo	Obrigatório	Chave	Regra	Descrição
id	UUID	Sim	PK		Identificador
usuario_id	UUID	Sim	FK	Deve existir usuário	Solicitante
equipamento_id	UUID	Sim	FK	Deve existir equipamento	Equipamento solicitado
administrador_id	UUID	Não	FK	Deve possuir perfil ADMIN	Responsável pela análise
status	ENUM	Sim		PENDENTE por padrão	Situação da solicitação
finalidade	TEXT	Sim		Não pode ser vazio	Justificativa do empréstimo
data_solicitacao	TIMESTAMP	Sim		Gerado automaticamente	Momento da criação

data_resposta	TIMESTAMP	Não	Apenas após análise	Momento da decisão
motivo_resposta	TEXT	Não		Observação da decisão
created_at	TIMESTAMP	Sim		Auditoria
updated_at	TIMESTAMP	Sim		Auditoria

Status

None

PENDENTE

APROVADA

REJEITADA

Regras

- Toda solicitação inicia como PENDENTE.
- Apenas ADMIN pode alterar o status.
- Uma solicitação aprovada ou rejeitada não pode retornar para PENDENTE.
- Apenas equipamentos ativos podem ser solicitados.
- O sistema deve verificar disponibilidade antes da criação da solicitação.

Entidade: EVENTO_HISTORICO

Campo	Tipo	Obrigatório	Chave	Regra	Descrição
id	UUID	Sim	PK		Identificador
solicitacao_id	UUID	Sim	FK	Deve existir solicitação	Solicitação relacionada

usuario_id	UUID	Sim	FK	Deve existir usuário	Responsável pelo evento
tipo_evento	ENUM	Sim		Evento do sistema	Tipo da alteração
descricao	TEXT	Não			Observações adicionais
created_at	TIMESTAMP	Sim		Gerado automaticamente	Data e hora do evento

Tipos de Evento

None

SOLICITACAO_CRIADA

SOLICITACAO_APROVADA

SOLICITACAO_REJEITADA

RETIRADA_REALIZADA

DEVOLUCAO_REALIZADA

OBSERVACAO

Regras

- Todo evento pertence a uma solicitação.
- Todo evento possui um responsável.
- Eventos não podem ser alterados ou removidos.

Entidade: EMPRESTIMO (Versão 1.1)

Campo	Tipo	Obrigatório	Chave	Regra	Descrição
id	UUID	Sim	PK		Identificador

solicitacao_id	UUID	Sim	FK	Deve existir solicitação aprovada	Origem do empréstimo
data_retirada	TIMESTAMP	Sim			Data da retirada
data_prevista_devolucao	DATE	Sim		Deve ser maior que data_retirada	Prazo de devolução
data_devolucao	TIMESTAMP	Não		Preenchido na devolução	Data de entrega
status	ENUM	Sim		ATIVO por padrão	Situação do empréstimo
created_at	TIMESTAMP	Sim			Auditoria
updated_at	TIMESTAMP	Sim			Auditoria

Status

None

ATIVO

DEVOLVIDO

ATRASADO

CANCELADO

Regras

- Apenas solicitações aprovadas podem gerar empréstimos.
- Uma solicitação gera, no máximo, um empréstimo.
- Um empréstimo ativo deve possuir data prevista de devolução.
- Ao registrar a devolução, o status muda para DEVOLVIDO.

Próximo documento

Na minha experiência, após o Dicionário de Dados, o documento mais importante é a **Especificação das Regras de Negócio por Entidade**. Em vez de listar regras de forma genérica, organizamos por entidade, por exemplo:

- **Solicitação:** regras de criação, aprovação, rejeição e transições de estado.
- **Equipamento:** regras de disponibilidade e estoque.
- **Usuário:** regras de autenticação e autorização.
- **Empréstimo:** regras de retirada, devolução e atraso.

Esse documento servirá como referência direta para implementar a camada de serviços (**Service**) no Spring Boot, pois praticamente cada regra de negócio documentada se transformará em um método ou validação da aplicação.

vamos para o postgres para fazer o desenvolvendo do create table do nosso banco

Perfeito. Eu gostaria de fazer apenas uma observação antes de começarmos a escrever SQL.

Estamos entrando na fase de implementação, então precisamos tomar algumas decisões de arquitetura que evitarão refatorações futuras.

Decisões que proponho

Banco

- PostgreSQL 16+
- UUID como chave primária
- **TIMESTAMPTZ** para datas e horas
- nomes em **snake_case**
- nomes das tabelas no singular (opcional, mas minha preferência)

Exemplo:

None

usuario

equipamento

categoria

solicitacao

evento_historico

emprestimo

Convenções

Toda tabela possuirá:

None

```
id UUID PRIMARY KEY
```

```
created_at TIMESTAMPTZ
```

```
updated_at TIMESTAMPTZ
```

Chaves primárias

Utilizaremos UUID.

None

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Isso evita problemas caso futuramente exista sincronização entre bancos ou APIs distribuídas.

Será necessário habilitar:

None

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Campos Enum

Ao invés de VARCHAR, eu utilizaria ENUM do PostgreSQL.

Exemplo

None

```
CREATE TYPE perfil_usuario AS ENUM (  
    'USER',  
    'ADMIN'  
);
```

Isso evita armazenar valores inválidos.

Auditoria

Utilizaremos

None

`created_at`

`updated_at`

em todas as tabelas.

Ordem de criação

Eu faria exatamente nesta sequência:

None

1. Extensões

2. ENUMs

3. Categoria

4. Usuário

5. Equipamento

6. Solicitação

7. Evento Histórico

8. Empréstimo (v1.1)

Assim nunca teremos problemas com Foreign Keys.

Outro detalhe importante

Eu adicionaria índices desde o início.

Por exemplo.

None
`email`

será pesquisado constantemente.

Logo

None
`UNIQUE(email)`

já cria um índice.

Já

None
`usuario_id`

na solicitação merece um índice.

Assim como

None
`status`

porque provavelmente teremos telas como

Mostrar todas as solicitações pendentes.

Minha sugestão

Ao invés de simplesmente gerar um único arquivo `create.sql`, proponho estruturarmos o banco como um projeto profissional usando migrações do **Flyway**, que já é a escolha natural quando utilizarmos Spring Boot.

A estrutura ficaria assim:

None

```
src/main/resources/db/migration/
```

```
V1__create_extensions.sql
```

```
V2__create_enums.sql
```

```
V3__create_categoria.sql
```

```
V4__create_usuario.sql
```

```
V5__create Equipamento.sql
```

```
V6__create_solicitacao.sql
```

```
V7__create_evento_historico.sql
```

```
V8__create_indexes.sql
```

As vantagens são:

- versionamento do banco junto com o código;
- criação automática do esquema ao iniciar a aplicação;
- histórico completo de alterações;
- facilidade para evoluir o banco (por exemplo, quando adicionarmos a tabela `emprestimo` na versão 1.1, bastará criar `V9__create_emprestimo.sql`).

Essa é uma prática amplamente utilizada em projetos Java com Spring Boot e vai agregar muito ao seu aprendizado. Minha sugestão é que construamos o banco dessa forma desde o início, em vez de manter um único script SQL gigante.



Chats with attachments paused

You've used all your messages with Instant until tomorrow at 1:13 AM. Upgrade to keep working with files and images here, or start a new chat to continue with a lower quality model.

[New chat](#)

[Claim free offer](#)